PLaND Path to Least Non Determinism

Maddipatla Naga Venkata Sai Krishna, **Asit Kumar Sahoo**

Affiliations: Both authors are Independent Researchers, San Francisco, CA, USA. Corresponding author: Maddipatla Naga Venkata Sai Krishna; mnvsk97@gmail.com.

Abstract

Path to Least Non Determinism (PLaND) is an evaluation-driven methodology for replacing suitable language-model work with code. It starts with an English standard operating procedure (SOP). A host reasoning agent inspects development examples and execution records, proposes a revised SOP package, and tests whether an executable step can reduce model use while preserving quality within a stated tolerance. Unresolved inputs use the unchanged English baseline. We collected new LEDGAR, CFPB, and SpamAssassin subsets with 500 development, 1,000 selection, and 500 reserved final-test cases each. Each reached split received three paired baseline/hybrid executions using hosted Gemini 3.5 Flash Lite. Selection required both arms to reach 80% accuracy, a paired accuracy-interval lower bound of at least −2 percentage points, and at least 5% fewer tokens with a positive interval lower bound. LEDGAR and SpamAssassin passed selection and final assessment in all repeats. Mean final-test accuracy across three runs changed from 94.93% to 95.80% for LEDGAR, with 74.54–74.58% fewer tokens. SpamAssassin mean accuracy changed from 97.67% to 97.20%, with 28.12–28.14% fewer tokens. CFPB hybrid mean selection accuracy was 79.27%; every run missed the floor, so its final test remained closed. Two provider-blocked selection emails were replaced under recorded amendments without reducing sample size. The study records package construction and evaluates frozen execution; it does not estimate autonomous discovery reliability across independent construction trials.

Keywords: agentic workflows, agent skills, SOP evolution, deterministic execution, language-model agents, token efficiency, hybrid systems

1 Introduction

A language model can interpret an unfamiliar input, but it may also spend tokens applying the same simple rule many times. A contract clause that explicitly names its governing law is one example. Can a workflow move such decisions into code without losing more accuracy than its users are willing to accept?

We present Path to Least Non Determinism (PLaND), a methodology for making this change through evaluation. It begins with an English standard operating procedure (SOP). A host reasoning agent studies development examples and execution records, proposes a revised SOP package, and measures it against the existing version. The revision may add an executable step while keeping the English instructions for inputs that the code cannot resolve. We call this combined version a hybrid SOP and the return to the model its fallback path.

PLaND uses existing agent infrastructure. Agent Skills packages instructions in a `SKILL.md` file with optional supporting files [1]. DeepAgents supports skills [2], and LangGraph provides workflow orchestration [3]. These are implementation components, not contributions introduced or independently benchmarked here.

Related work improves model programs, workflows, and skills. DSPy optimizes language-model pipelines against task metrics [4]. AutoFlow, AFlow, and Automated Design of Agentic Systems search over workflows or agent designs [5–7]. SkillOpt and SkillRevise revise reusable skills using evaluation feedback [8, 9]. SkillReducer studies token-efficient skill representations [10], while ACES evaluates the effect of skills in paired agent trials [11]. PLaND focuses on replacing suitable model-mediated work with explicit computation while evaluating the whole workflow, including fallback. We do not compare against these methods experimentally or claim that the general idea of hybrid rules and models is new.

This paper evaluates fixed English and hybrid SOP execution on legal clauses, consumer complaints, and email. It also records how the packages were constructed. The repeated executions test the selected packages, not the reliability of independently repeating construction. Reducing non-determinism here means reducing dependence on model calls; it does not mean proving lower output variability or finding a globally optimal workflow.

2 Material and Methods

2.1 Starting with an English SOP

PLaND has two methodology skills: `generate-initial-version` and `pland-evolver`. The first creates an initial agent with one English SOP, approved data access, and a task-specific evaluator. The second directs a host reasoning agent to measure that SOP, inspect development records, and propose a bounded revision. Collection control, retries, and evidence packaging are separate experiment operations.

A skill directory contains its main instructions in `SKILL.md`. It may also contain supporting instructions in `references/`, code in `scripts/`, and reusable files in `assets/` [1]. Adding a reference file does not make a step deterministic. A step becomes executable only when the workflow runs code and uses its result.

The allowed labels are part of the task definition. We selected existing dataset categories before collection and supplied that vocabulary to both the model and the Python classifier. PLaND did not discover these categories. Each case also has an expected label held by the evaluator. That case-specific answer is not supplied to either classifier.

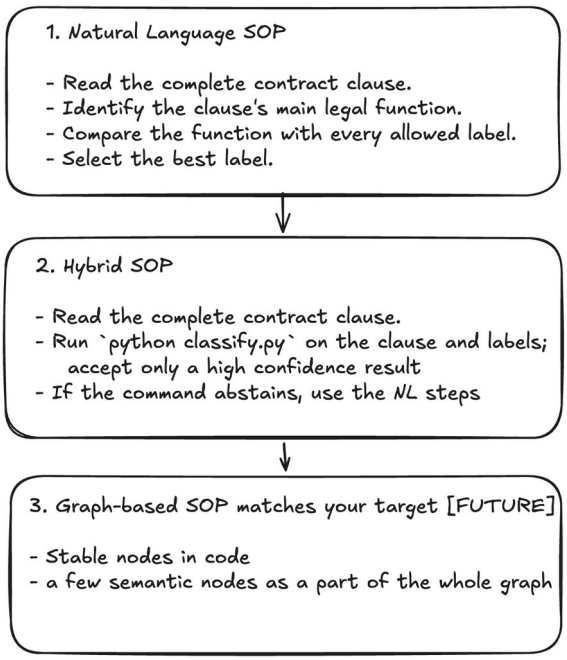

Figure 1. From English instructions to selective code execution. The first two stages illustrate the evaluated design. The figure's phrase "high confidence" refers to support from development evidence; the tested scripts do not compute or enforce a numerical confidence score. They use ordered rules and input checks, described in Section 3.2. Graph-based workflows are a future direction, not a measured result.

2.2 Proposing and checking a candidate

A candidate is a revised version of the whole SOP package, including its instructions and supporting code. It is not one dataset row or one model answer. A candidate makes one bounded change, such as adding or narrowing a rule-based classification step. That step can contain several related patterns.

The host agent chooses what to propose by inspecting development inputs, labels, errors, and execution traces. It may revise an English instruction when interpretation is still needed, or write Python or Bash for a repeatable operation with explicit input and output conditions. This choice is a proposal, not proof that the replacement is safe. Evaluation decides whether the complete candidate is acceptable.

For example, consider the illustrative clause "This Agreement shall be governed by the laws of the State of California." Governing Laws is an existing LEDGAR category. If phrases of this kind recur in development records, the host agent can propose a matching rule. The script returns the category when its conditions hold; otherwise the baseline model reads the clause. The evaluator compares the returned label with the hidden dataset label. Success on this example does not establish that the rule works on other clauses.

We use three separate groups of cases. Development cases guide revisions. Selection cases decide whether one frozen candidate is accepted. Final-test cases assess the selected package after acceptance. The collection follows this sequence:

1. Measure the English baseline on development cases. Refine it only on development evidence until it reaches the readiness floor or its attempt limit.
2. Inspect development records and propose a candidate. Save its hypothesis, instructions, code, and measurements.
3. Apply the development checks in Section 2.3. If the candidate fails and attempts remain, revise it using development evidence. Stop if no suitable change is found or the budget is exhausted.
4. Freeze the first development-ready candidate and compare it with the baseline on selection cases. Selection rejection ends that dataset's study; it does not start another candidate search.
5. After selection acceptance, run the reserved final test without changing either package. Report the final result, including a failure if the final criteria are not met.

The approved plan allowed at most ten baseline attempts and ten hybrid attempts. These are practical search limits chosen for this study, not a theoretically optimal budget. Only one hybrid per dataset could enter selection. Repeated executions of that hybrid did not count as new candidate attempts.

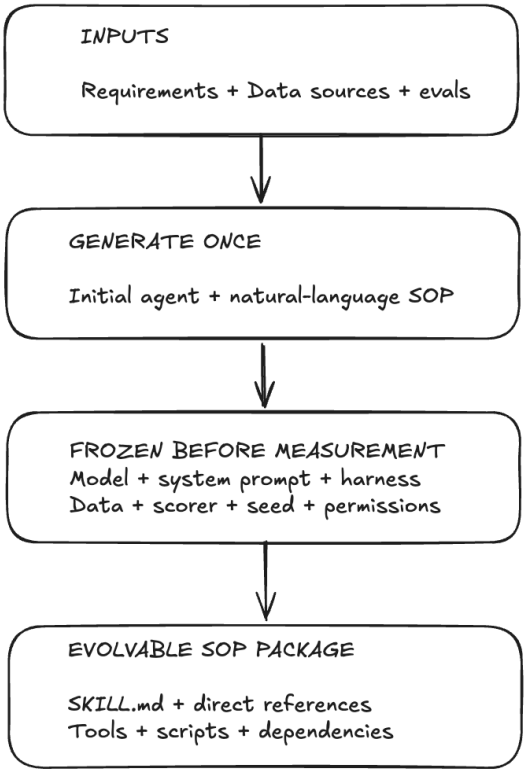

Figure 2. What stays fixed during a comparison. Candidate edits are confined to the SOP package. Model identity, cases, system prompt, evaluator, and runtime settings stay fixed. "Seed" includes data sampling and bootstrap sampling; this hosted provider did not support a model-inference seed. SHA-256 hashes identify exact file contents so later changes can be detected.

2.3 Development readiness

The preliminary checks prevent a weak or invalid package from consuming the held-out selection set. The English baseline must first reach 80% development accuracy. A hybrid must then reach 80% development accuracy, have no recorded execution errors, reduce total model tokens by at least 5% against the paired baseline, and satisfy the frozen-input and SOP contracts. The hybrid contract requires an executable step and preserved English fallback. The selected candidate must pass its primary development assessment and all three development-repeat assessments.

These checks screen proposals on data already available to the host agent. They are not the final acceptance test. Development assessment uses observed accuracy and token improvement; the paired bootstrap requirements in Section 3.3 apply at selection. Selection and final-test observations cannot guide revisions.

The saved construction trail contains hypotheses, development pattern analyses, rejected attempts, package files, and decisions. The host agent narrowed LEDGAR's notice and survival patterns after development errors, restricted CFPB routing to four product patterns, and added email phrase/header rules for SpamAssassin. This records the construction performed in the study. It does not estimate how often an autonomous agent would discover useful rules from a new starting point.

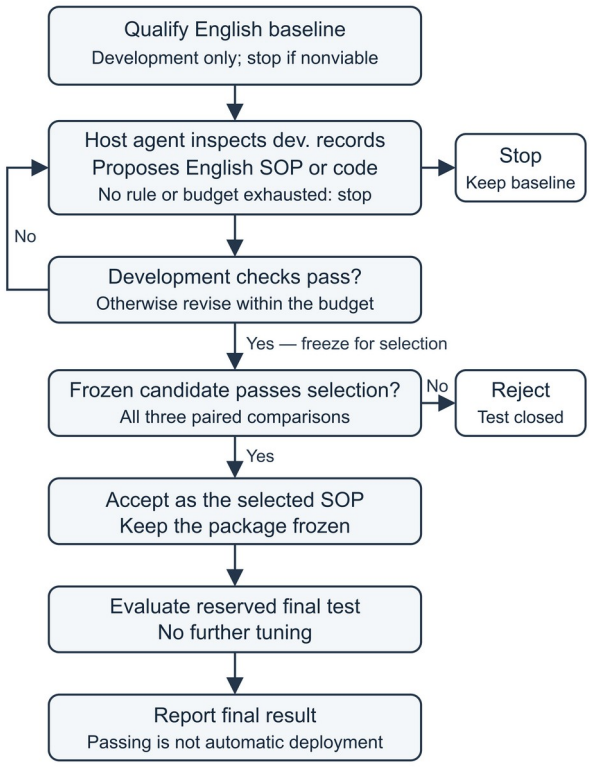

Figure 3. Proposal, acceptance, and final assessment. The host reasoning agent generates a candidate from development records. Only the development loop can produce another candidate. Selection rejection ends the study. Selection acceptance fixes the new SOP before reserved-test assessment and reporting; it is not automatic permission to deploy it.

3 Experimental Setup

3.1 Data and split sizes

We collected fresh subsets from LEDGAR through LexGLUE [14, 15], the CFPB Consumer Complaint Database [16], and the SpamAssassin public email corpus [17]. The tasks identify a clause's legal category, a complaint's product category, and whether an email is ham or spam. LEDGAR and CFPB each use ten existing labels; SpamAssassin uses two. Appendix C lists them.

Table 1. Cases prepared for each dataset

Dataset	Dev.	Selection	Final test
LEDGAR	500	1,000	500
CFPB	500	1,000	500
SpamAssassin	500	1,000	500

All splits are balanced within the chosen labels and do not represent natural label frequencies. Preparation excluded previously opened identifiers and normalized-content duplicates, then checked for overlap among the new splits. All LEDGAR study splits came from the source training partition. These are new study splits, not the native LexGLUE benchmark partitions; we make no claim of a standard LexGLUE benchmark score.

Dev. means development. The approved 500/1,000/500 allocation gives more cases to the decision that controls whether a candidate advances, while retaining separate development and final-test data. It is a study design choice, not a universal recommended ratio. Development can still miss rare patterns, and these sizes do not establish enough statistical power for every application.

3.2 Execution and scoring

The baseline sends each input, the allowed labels, and its complete English SOP to the model. The hybrid first calls a Python classification function directly from the runner. It uses ordered regular-expression rules and returns the first matching allowed label with a rule identifier. It does not detect all possible category conflicts and then abstain. No match, an invalid input, or an invalid command result triggers fallback; command exceptions are separately recorded as errors.

LEDGAR rules recognize category-specific phrases. CFPB rules cover Prepaid card, Student loan, Mortgage, and the selected payday/personal-loan category. The email script checks spam phrases such as “removed from” and “viagra,” then ham reply headers. Rule order is part of the frozen package. A valid rule result completes the case without a model call. The harness executes this routing; the benchmark does not ask an autonomous model to select tools or launch a shell for each case.

Fallback sends the same case through the complete, unchanged baseline SOP with the same model and system prompt. It does not use a shorter substitute prompt. The saved SOP contract and corresponding input-token counts check this property. Both arms use the same label-only JSON format, `{"label":"<allowed label>"}`, in primary runs and repeats. There is no confidence field.

After JSON parsing, the scorer compares the returned label with the expected label by literal string equality. It does not normalize capitalization, spaces inside the label, punctuation, or aliases. JSON formatting whitespace does not affect the decoded string. A matching label scores one; an incorrect or invalid answer scores zero. Accuracy is correct cases divided by all evaluated cases. Parsing and execution errors are recorded separately and cannot satisfy a zero-error release requirement.

We count model input plus output tokens reported by the provider. Completion tokens already include any reported reasoning tokens; they are not counted twice. Cached input tokens remain part of input-token use. Code-resolved cases use zero model tokens. Execution totals exclude the host agent's construction work and operational costs of failed attempts, retries, and provider screening. They are not total project cost, dollar savings, or energy measurements.

3.3 Selection criteria and uncertainty

The protocol fixed the same acceptance criteria for all datasets before selection. Every paired selection execution had to satisfy all of the following:

1. Baseline and hybrid accuracy are both at least 80%.
2. The lower endpoint of the paired 95% accuracy-difference interval is at least −2 percentage points.
3. Total model-token use is reduced by at least 5%.
4. The lower endpoint of the paired 95% token-reduction interval is greater than zero.
5. The evaluated outputs are complete and have no execution errors.

These thresholds are the authors' engineering choices. They are not supplied by dataset creators, justified as deployment standards, or derived from error costs. Non-inferiority provides a framework for evaluating an allowed loss [13]; it does not determine which loss an application should accept.

To calculate the intervals, we keep the baseline and hybrid results for each case together. We repeatedly draw a new sample of these pairs, with replacement, using the original split's case count. For each sample, we calculate hybrid accuracy minus baseline accuracy and the fraction of baseline tokens saved. After 5,000 samples, we take the 2.5th and 97.5th percentiles as the interval endpoints [18]. This paired bootstrap preserves the shared input and avoids assuming a normal distribution for token use.

A lower accuracy endpoint above −2 points keeps the estimated loss within the chosen tolerance under this procedure. A positive lower token endpoint supports a reduction rather than an increase. Neither interval guarantees future performance. The bootstrap treats cases as exchangeable; related clauses or emails may violate that assumption. We report per-repeat intervals, not a pooled interval that treats repeated views of the same cases as independent data.

The same criteria assess the final test after selection acceptance. A final-test failure would be reported without further tuning. It would not authorize another candidate search.

3.4 Model configuration and repeats

The evaluated model was `google/gemini-3.5-flash-lite`, accessed through OpenRouter with Google AI Studio as the sole permitted provider endpoint. The setup used DeepAgents 0.7.12, low reasoning effort, a 1,024-token completion limit, non-streaming requests, and a 300-second timeout. Temperature was omitted, leaving the service default. Provider fallback was disabled. The configuration hash identifies request and routing settings, not model weights. Hosted weight bytes and a stable system fingerprint were unavailable.

The runner used eight concurrent case workers, preserving the frozen setting between arms. HTTP 429 responses were recorded and retried, honoring `Retry-After` or using exponential backoff with case-specific jitter and a maximum delay of 300 seconds. They were not classified as failed examples. Interrupted runs retained completed case receipts and resumed missing work after checking frozen inputs and configuration.

Three paired executions ran on every reached split with the same cases, SOPs, label format, and runtime. Their identifiers are 20260903, 20260904, and 20260905. Although the plan calls them repeat seeds, the adapter records `seed_supported: false` and sends no inference seed. They are repeat identifiers, not controlled model-randomness settings. Data sampling and bootstrap sampling used seed 20260902; the token bootstrap used that value plus one. We report each held-out result without assuming deterministic service behavior.

3.5 Provider blocks and amendments

Two SpamAssassin selection emails produced terminal provider content blocks. Under two recorded, author-approved amendments, each was replaced with a unique email of the same label from the seeded reserve. Provider safeguards were not weakened and blocked text was not rewritten to force an answer. Screening checked request acceptability, not classification correctness.

Each amendment preserved 1,000 balanced selection cases and restarted all three selection pairs on the amended set. Development and reserved final-test cases, SOPs, model settings, and thresholds remained unchanged. Original blocked attempts and superseded selection evidence were retained separately. Two later connection interruptions resumed from saved receipts; all reported completed outputs have zero errors.

Replacement preserves sample size but changes the selection population to provider-processable content. It may introduce selection bias and does not establish performance on blocked emails. Screening also exposed some selection inputs to the provider before paired runs; its responses were not used to tune rules.

4 Results

4.1 Development and selection

Every English baseline reached readiness on its first attempt. LEDGAR and CFPB each rejected one hybrid development candidate before qualifying the second. SpamAssassin qualified its first hybrid. Table 2 reports development repeats of the qualified packages; primary

construction assessments and unsuccessful attempts remain in the evidence.

Table 2. Development attempts and mean accuracy

Dataset	B attempts	H attempts	Mean accuracy B → H (%)
LEDGAR	1	2	96.53 → 97.20
CFPB	1	2	81.67 → 81.87
SpamAssassin	1	1	98.87 → 98.67

In Tables 2–4, B is the English baseline and H is the hybrid. Mean accuracy is the sum of the three per-run accuracies divided by three, using unrounded values. The repeats use the same cases; averaging does not increase the number of independent cases. Token savings are shown as the minimum–maximum across runs, not confidence intervals. Each selection run used 1,000 cases per arm.

Table 3. Selection results across three paired executions

Dataset	Mean accuracy B → H (%)	Tokens saved (%)	Decision
LEDGAR	97.23 → 96.83	70.29–70.30	Accept
CFPB	79.73 → 79.27	23.42–23.47	Reject
SpamAssassin	98.93 → 98.03	32.15	Accept

LEDGAR and SpamAssassin passed every selection requirement in each repeat, permitting final testing. CFPB was rejected. We did not average away a failed gate or select only the most favorable repeat.

4.2 LEDGAR final test

Across the three final-test executions, mean baseline accuracy was 94.93%, and mean hybrid accuracy was 95.80%. The rules answered 370 of 500 cases (74.0%) without a model call. Rule accuracy on those cases was 95.9%. Total model-token use fell 74.54–74.58%. All three paired accuracy intervals stayed within the allowed lower bound; their lower endpoints were -0.4, -0.6, -0.6 percentage points. Every final-test comparison passed the recorded criteria.

The result supports selective routing for this balanced ten-category clause subset. It does not show that the rules are correct on every routed case, on the full LEDGAR label set, or on clauses from another source distribution.

4.3 CFPB selection rejection

CFPB failed the absolute accuracy floor in every selection repeat. Across the three runs, mean baseline accuracy was 79.73%, and mean hybrid accuracy was 79.27%. The hybrid never reached the required 80%, despite reducing tokens by 23.42–23.47%. Its paired accuracy intervals met the −2-point requirement, and its token intervals were positive. Those relative improvements could not compensate for failing the absolute floor. Selection rejection was terminal: no replacement candidate was created and the 500-case final test was not opened.

The baseline itself was near the floor and fell below it in two repeats. This left little accuracy headroom under the fixed conditions. The result rejects this workflow under the protocol, not the possibility of useful rules for complaint classification.

4.4 SpamAssassin final test

Across the three final-test executions, mean baseline accuracy was 97.67%, and mean hybrid accuracy was 97.20%. The rules answered 177 of 500 cases (35.4%) without a model call. Rule accuracy on those cases was 97.7%. Total model-token use fell 28.12–28.14%. All three paired accuracy intervals stayed within the allowed lower bound; their lower endpoints were -1.6, -1.6, -1.4 percentage points. Every final-test comparison passed the recorded criteria.

SpamAssassin's hybrid had lower observed accuracy than the baseline in every selection and final-test repeat. It passed because the estimated loss stayed within the predeclared tolerance, not because accuracy improved. The old public corpus and provider-block replacements limit generalization to current email.

Table 4. Reserved final-test results across three paired executions

Dataset	Mean accuracy B → H (%)	Calls B → H	Tokens saved (%)
LEDGAR	94.93 → 95.80	500 → 130	74.54–74.58
SpamAssassin	97.67 → 97.20	500 → 323	28.12–28.14

Both final tests used 500 cases per arm in each repeat. CFPB has no final-test result. Appendix A reports all held-out repeats and their intervals individually.

4.5 Where token savings came from

Fallback input-token counts matched the baseline on every corresponding final-test case. In LEDGAR final-test repeat 1, the hybrid saved 180,904 tokens: 180,828 from avoiding model calls and 76 from shorter model outputs on fallback cases. In SpamAssassin final-test repeat 1, the corresponding amounts were 412,015, 411,938, and 77 tokens. The measured savings therefore came almost entirely from bypassing model calls. They did not come from shortening the fallback prompt. For one SpamAssassin selection email, provider-reported input counts differed by 1, 3, 2 tokens across the three pairs despite the unchanged prompt-construction contract. The saved receipts do not explain this small discrepancy; it is retained in the reported totals.

Coverage alone does not determine token reduction: long inputs use more tokens than short ones. A rule bypassing many short emails may save a smaller fraction of tokens than its fraction of cases. The paired comparison captures this difference and any output variation on fallback cases.

5 Discussion

The results show acceptance and rejection under the same protocol. LEDGAR and SpamAssassin reduced model use within the chosen quality limits. CFPB reduced tokens but missed the absolute accuracy floor. A token-saving change is therefore not acceptable merely because its accuracy is close to the baseline.

PLaND makes the boundary between code and model reasoning explicit and testable. The model handles inputs that code declines; the evaluator judges the final answer from either path. Unchanged fallback makes this comparison easier to interpret than a package that changes rules and shortens prompts together. Still, a deterministic path is only as reliable as its conditions and development evidence. First-match rules can make incorrect decisions when an input contains several relevant patterns.

The construction trail demonstrates one application of the skills per dataset, including development revisions. Execution repeats do not establish the probability that a host agent finds a good rule, the number of independent searches needed, or their cost. Those questions need repeated construction trials on separate data with a recorded host-model configuration. Gemini is the runtime classifier here, not a benchmark of the host agent's reasoning ability.

These are narrow, balanced classification subsets. We did not compare with a trained lightweight classifier, evaluate a complete business process, or conduct deployment-specific risk analysis. The hosted endpoint can also change behind an unchanged model name. Saved settings and receipts make these comparisons auditable but cannot guarantee bit-for-bit future regeneration.

Token use measures model inference rather than all computational expense. Construction, screening, retries, local computation, and maintenance are outside these execution totals. We therefore do not infer dollar savings, energy savings, or production latency gains. Resumed-run wall times cover only part of execution and are not used as headline results.

6 Future Work

The next evaluation should repeat package construction from independently prepared development sets and measure success rate, failed proposals, and construction cost. This would test whether the skills reliably find useful replacements. Comparisons with trained small classifiers and other workflow-optimization methods would help identify when the methodology is useful.

Broader tests should include unseen organizations or time periods, rare classes, and tasks beyond classification. Deployment would require error-specific quality requirements, privacy review, and monitoring for data or model changes. Organizing accepted steps into a workflow graph is a possible extension, illustrated in Figure 1, but remains unevaluated.

7 Conclusion

PLaND provides a process for starting with English instructions, proposing executable replacements from development evidence, and accepting them only after a separate quality-and-token check. In this study, LEDGAR and SpamAssassin passed selection and final assessment across all three paired executions, reducing final-test model tokens by 74.54–74.58% and 28.12–28.14%, respectively. CFPB failed the selection accuracy floor and did not proceed to final test. These results show that selective code execution can reduce model use under explicit quality limits, but that useful-looking rules do not always produce an acceptable workflow. Independent tests of the construction process and broader tasks are needed before making stronger claims.

Acknowledgements and Disclosures

The authors used AI-assisted tools for coding, experiment support, and manuscript preparation. The authors are responsible for the reported results and final text. No external funding was received, and the authors declare no competing interests.

Data and Code Availability

The public companion repository [12] contains the skills, frozen plans, packages, case-level outputs, comparisons, command ledgers, and manifests. The collection snapshot is commit `4ad24c9838922b3db777ac4ba4dbc34de7a449ae`. The main branch also contains the reviewed paper, its audit, and portable manifests that identify the exact evidence. Appendix B gives paths and verification commands. Reference [12] identifies author-produced artifacts, not independent support for the methodology.

Raw inputs remain outside the repository and are subject to source terms. Exact CFPB regeneration requires the saved local API snapshot; a fresh download is a different input collection. Case outputs support offline recomputation without those texts or new model calls. Inference reruns require frozen inputs and service access, and no hosted weight digest is available. File verification, statistical recomputation, and inference regeneration are distinct operations.

Appendix A Individual held-out executions

Repeats 1, 2, and 3 correspond to identifiers 20260903, 20260904, and 20260905. Each pair uses identical cases. Repeats are not additional independent samples and are not pooled to enlarge the case count. “Test” means reserved final test.

Table A1. Accuracy and total model tokens by repeat

Dataset and split	Run	Accuracy B → H (%)	Tokens B → H
LEDGAR selection	1	97.0 → 96.7	472,670 → 140,424
LEDGAR selection	2	97.3 → 96.9	472,717 → 140,392
LEDGAR selection	3	97.4 → 96.9	472,800 → 140,484
LEDGAR test	1	94.8 → 95.8	242,557 → 61,653
LEDGAR test	2	95.0 → 95.8	242,421 → 61,700
LEDGAR test	3	95.0 → 95.8	242,372 → 61,707
CFPB selection	1	80.0 → 79.4	700,508 → 536,424
CFPB selection	2	79.6 → 78.9	700,526 → 536,141
CFPB selection	3	79.6 → 79.5	700,470 → 536,111
SpamAssassin selection	1	99.0 → 97.9	2,636,096 → 1,788,601
SpamAssassin selection	2	98.8 → 98.1	2,636,096 → 1,788,658
SpamAssassin selection	3	99.0 → 98.1	2,636,117 → 1,788,728
SpamAssassin test	1	97.8 → 97.2	1,464,817 → 1,052,802
SpamAssassin test	2	97.4 → 97.0	1,465,131 → 1,052,891
SpamAssassin test	3	97.8 → 97.4	1,464,835 → 1,052,948

Table A2. Paired bootstrap intervals by repeat

Dataset and split	Run	Accuracy 95% interval (points)	Token saving 95% interval (%)
LEDGAR selection	1	[-1.10, 0.50]	[67.38, 73.16]
LEDGAR selection	2	[-1.20, 0.40]	[67.40, 73.17]
LEDGAR selection	3	[-1.30, 0.30]	[67.39, 73.15]
LEDGAR test	1	[-0.40, 2.60]	[70.65, 78.36]
LEDGAR test	2	[-0.60, 2.20]	[70.62, 78.34]
LEDGAR test	3	[-0.60, 2.20]	[70.61, 78.33]
CFPB selection	1	[-1.60, 0.40]	[20.56, 26.33]
CFPB selection	2	[-1.80, 0.40]	[20.60, 26.38]
CFPB selection	3	[-1.20, 1.10]	[20.60, 26.38]
SpamAssassin selection	1	[-1.80, -0.50]	[28.01, 36.63]
SpamAssassin selection	2	[-1.40, -0.10]	[28.01, 36.63]
SpamAssassin selection	3	[-1.60, -0.30]	[28.00, 36.63]
SpamAssassin test	1	[-1.60, 0.40]	[22.13, 35.18]
SpamAssassin test	2	[-1.60, 0.60]	[22.14, 35.18]
SpamAssassin test	3	[-1.40, 0.60]	[22.13, 35.17]

Accuracy intervals are H minus B in percentage points. Token intervals are percentage reductions relative to B. Calculations use unrounded values. CFPB met the relative interval requirements but failed the absolute accuracy requirement.

Appendix B Evidence and verification

The collection root is `experiments/gemini-3.5-flash-lite/`. The base plan and protocol are in `2026-09-05-11-41-pm/`. The base plan SHA-256 is `57f613822481e0d95cecba0105fe227b96476c70ea9568848f0f0bcc0934eccd`. SpamAssassin's final selection uses `plan-amendment-02.json` in that directory, SHA-256 `35c4c89d99f7cd788a38f2e36ea46ca75beb58ee9e5fcab44019b5a66036bea7`.

LEDGAR's decisions and comparisons are in `2026-09-05-11-41-pm/ledgar-valid/`; its complete outputs have the `valid-` prefix in adjacent `ledgar/results/`. CFPB's records are in `2026-09-05-11-41-pm/cfpb/`. SpamAssassin's development and blocked original selection are in `2026-09-05-11-41-pm/spamassassin/`, its superseded first amendment is in `2026-09-06-12-59-am/spamassassin/`, and its completed selection and final test are in `2026-09-06-01-10-am/spamassassin/`. Each packaged controller has an `evidence-manifest.json`. Distinct conditions are retained separately.

The manuscript audit is in `2026-09-06-manuscript/`. Its `paper-calculations.json` identifies every reported pair, file hash, SOP hash, classifier hash, gate, and recomputed interval. `evidence-files.json` inventories saved collection and code files; `paper-artifacts.json` identifies manuscript outputs. The provenance addendum records superseded metadata issues without changing frozen results.

Run `uv sync --project reproduce --frozen`, then `reproduce/.venv/bin/python reproduce/verify.py`. Run `reproduce/.venv/bin/python paper/audit_paper.py --artifacts` to recompute results and verify manuscript files. These commands are offline and do not open CFPB's reserved test. The audit derives accuracy and tokens from case outputs, independently rebuilds paired bootstrap intervals, and checks saved comparisons. Command ledgers retain original collection and retry commands.

Appendix C Labels and sources

LEDGAR uses Governing Laws, Counterparts, Notices, Entire Agreements, Severability, Amendments, Survival, Assignments, Expenses, and Terms. These are corpus categories [14] distributed through LexGLUE [15]. All three study splits were newly sampled from the source training partition after prior-case exclusions.

CFPB uses the product field associated with published narratives in the saved snapshot [16]. The labels are Mortgage; Checking or savings account; Student loan; Money transfer, virtual currency, or money service; Vehicle loan or lease; Prepaid card; Payday loan, title loan, personal loan, or advance loan; Credit card; Debt collection; and Credit reporting or other personal consumer reports. These strings come from the frozen snapshot, not label discovery.

SpamAssassin uses ham from easy-ham and hard-ham archives and spam from spam and spam-2 archives dated 20030228 [17]. In its final selection set, blocked spam messages `mail-3125dcdc7726abc1c128` and `mail-449598896e25273135c3` were replaced by `mail-023db858450b084e7238` and `mail-0ae0fc25ca50549e73a8`, respectively. The set retained 500 ham and 500 spam cases. Data audits record source hashes, case identifiers, label balance, and duplicate/overlap checks.

References

[1] Agent Skills. Specification. Official documentation. Accessed September 6, 2026.

[2] LangChain. Skills in DeepAgents. Official documentation. Accessed September 6, 2026.

[3] LangChain. LangGraph overview. Official documentation. Accessed September 6, 2026.

[4] Khattab O, et al. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. 2023. arXiv:2310.03714.

[5] Li Z, et al. AutoFlow: Automated Workflow Generation for Large Language Model Agents. 2024. arXiv:2407.12821.

[6] Zhang J, et al. AFlow: Automating Agentic Workflow Generation. 2024. arXiv:2410.10762.

[7] Hu S, Lu C, Clune J. Automated Design of Agentic Systems. 2024. arXiv:2408.08435.

[8] Yang Y, et al. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. 2026. arXiv:2605.23904.

[9] Liu Y, et al. SkillRevise: Improving LLM-Authored Agent Skills via Trace-Conditioned Skill Revision. 2026. arXiv:2606.01139.

[10] Gao Y, et al. SkillReducer: Optimizing LLM Agent Skills for Token Efficiency. 2026. arXiv:2603.29919.

[11] Kevin C, et al. Evaluating Skills, Not Just Agents: Agentic Continuous Evaluation of Skills. 2026. arXiv:2608.20614.

[12] Maddipatla Naga Venkata Sai Krishna, Asit Kumar Sahoo. PLaND experiment records and implementation. Companion research artifact. 2026. Public repository. Frozen collection. Exact plan, evidence, and audit paths are given in Appendix B. Author-produced provenance, not independent validation.

[13] Walker E, Nowacki AS. Understanding Equivalence and Noninferiority Testing. Journal of General Internal Medicine. 2011;26(2):192–196. doi:10.1007/s11606-010-1513-8.

[14] Tuggener D, von Däniken P, Peetz T, Cieliebak M. LEDGAR: A Large-Scale Multi-label Corpus for Text Classification of Legal Provisions in Contracts. Proceedings of LREC. 2020:1235–1241. ACL Anthology.

[15] Chalkidis I, et al. LexGLUE: A Benchmark Dataset for Legal Language Understanding in English. Proceedings of ACL. 2022:4310–4330. ACL Anthology.

[16] Consumer Financial Protection Bureau. Consumer Complaint Database. Official dataset. Accessed September 6, 2026.

[17] Apache SpamAssassin. Public mail corpus. Official dataset. Accessed September 6, 2026.

[18] Efron B. Bootstrap Methods: Another Look at the Jackknife. The Annals of Statistics. 1979;7(1):1–26. doi:10.1214/aos/1176344552.